



Projet de Fin d'Études:

PICO-CENTRALE HYDROÉLECTRIQUE

en vue de l'obtention du

Brevet de Technicien Supérieur de

Cybersécurité ,Informatique et Réseaux ,Électroniques

Spécialité : Électroniques et Réseau

PARTIE 3 : Communication LoRa

Etudiant 3 : Matheis MICHALAK

SOMMAIRE

Remerciements :	3
Contexte :	4
1. Travail d'équipe :	5
1.1. Espace Collaboratif :	5
1.2. Stockage en Ligne:	5
1.3. Versionning :	5
2. Planification :	7
2.1. Planification Prévisionnelle :	7
2.2. Planification Réelle :	8
3. Technologies Utilisé :	10
3.1. Pourquoi ce module ?	10
3.2. Explications Techniques :	10
3.3. Création et Câblage d'un Module :	12
4. Objectif :	14
5. Principe de Communication :	15
5.1. Communication Simplex :	15
5.2. Communication Full-Duplex :	15
5.3. Communication Half-Duplex (Final) :	15
6. Élément de la Trame :	17
6.1. Trame d'Information :	17
7. Manipulation du Systèmes :	18
7.1. Règle de sécurité :	18
7.2. Opération de Contrôle :	18
7.3. Opération de Maintenance :	18
Annexes:	19
1. Code sous MicroPython:	19
2. Algorithme :	23
3. Gestion de la Trame :	27

Remerciements :

Je tiens tout d'abord à exprimer ma profonde gratitude envers l'association Viseta pour nous avoir accordé sa confiance et permis de participer activement à la construction de ce projet stimulant. Cette expérience nous a offert l'opportunité de confronter nos connaissances théoriques aux réalités du terrain.

Mes remerciements s'adressent également à l'ensemble du corps professoral de la section BTS CIEL. Merci pour votre accompagnement pédagogique, votre disponibilité et vos précieux conseils qui nous guident tout au long de ces deux années de formation et nous soutiendront jusqu'à l'échéance de notre diplôme.

Enfin, je tiens à remercier chaleureusement les membres de mon équipe projet ainsi que toute la promotion 2024-2026 du BTS CIEL du lycée Bourdelle. Un grand merci pour votre implication, l'esprit d'entraide et la cohésion dont tout le monde a fait preuve, sans oublier les éclats de rire et les nombreux moments mémorables partagés ensemble au quotidien.

Contexte :

Les tâches qui me sont données en tant qu'Étudiant n°3 sont de mettre en place un protocole de communication entre le contrôleur local et le contrôleur principal à l'aide d'un module lora .

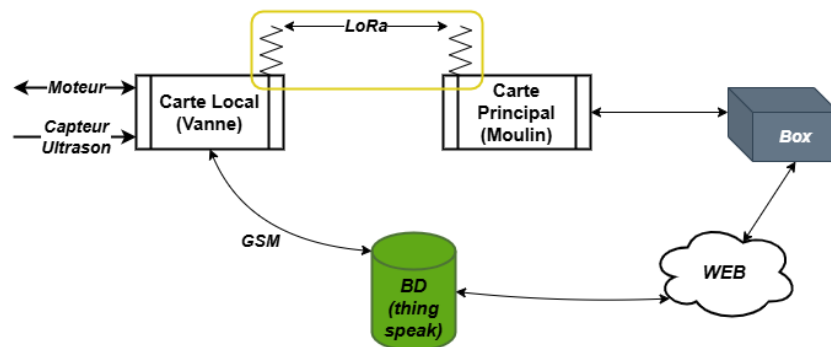
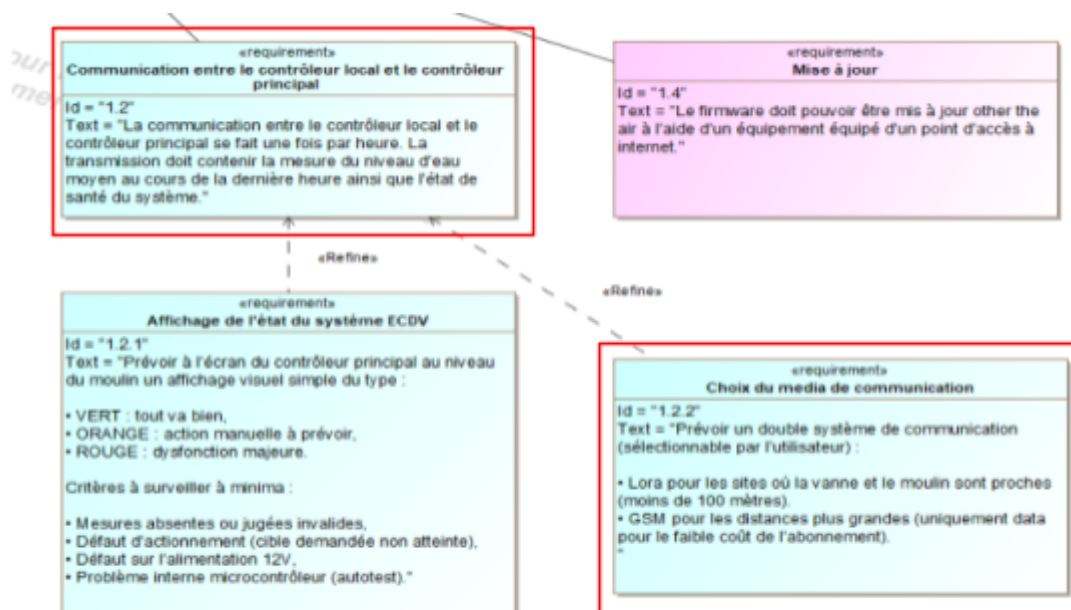


Figure 1: Synoptique du système

Bien évidemment ma partie comprend la création du composant pour le schéma électrique ainsi que son empreinte Ares ,du code pour l'établissement du protocole de communication et les tests associés

L'exigences principale de ma partie est "Communication entre le contrôleur local et le contrôleur principal" de lequel découle le choix du média de communication



1. Travail d'équipe :

1.1. Espace Collaboratif :

Mon travail se trouve essentiellement dans la branche "3 Communication Lora" car je me concentre en grande partie sur cette tâche dans le projet. Jessy ,mon camarade ,et moi-même avons en majeure partie œuvré à la mise en place de cet outil pour le suivi des avancées ,autant pour l'équipe du projet que pour les professeurs surveillants.

1.2. Stockage en Ligne:

Tous nos codes ,schémas ,photos ,rapports et ressources en tout genre sont rangés dans le Github selon l'utilisation de chacun. J'essaie de tenir ma branche le plus propre possible autant pour moi que pour mes camarades et professeurs.

1.3. Versionning :

Pour avancer efficacement et ne pas me perdre dans mes codes ,je nomme mes versions en fonction de l'avancement de cette dernière ainsi je ne me trompe pas quand il s'agit d'aller chercher des fonctions ou informations dans des codes antérieurs.

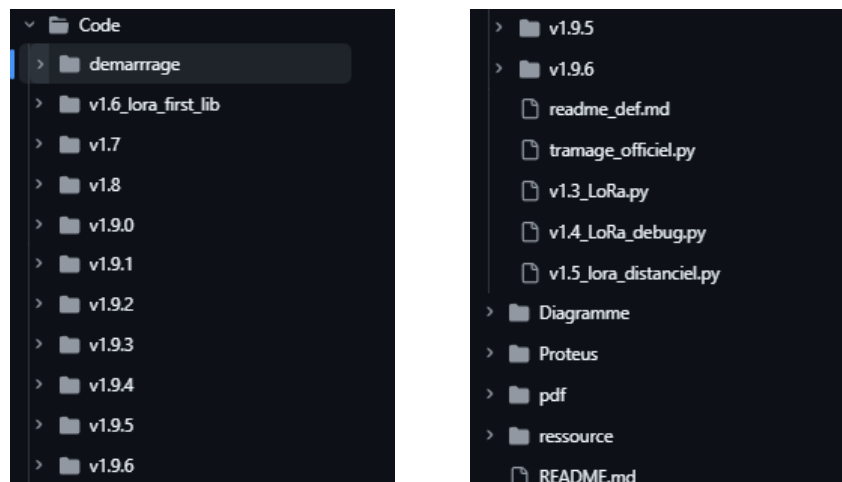


Figure 2: Arborescence de la branche 3 Communication LoRa

En plus de tout cela ,le fait de passer mon code d'un simple algorithme a un programme et sa librairie aurait vite donné beaucoup de fichiers avec le

même nom car la bibliothèque ne change pas radicalement entre chaque séance de programmation ,l'essentiel des définitions ajoutées au code se fait d'abord dans le fichier principal et transposé par la suite dans la bibliothèque pour libérer des lignes et augmenter la lisibilité du programme ,d'où l'utilisation de fichier versionné afin de ranger tout mon travail.

On peut voir par ailleurs sur la figure 2, le moment où je suis passé du code unique au code orienté objet avec le dossier "v1.6 lora first lib" qui contient la v1 de la librairie ainsi que la v1 du full duplex , dorénavant obsolète.

2. Planification :

2.1. Planification Prévisionnelle :

La planification prévisionnelle d'un projet n'est pas nécessaire à la conception du projet en lui-même mais plus pour guider sa gestion. Les retards ou les avances ,lié ou non à l'équipe ou aux fournisseurs

J'ai réparti les tâches de la manière suivante :

Planification	Comprend toute la partie organisation d'équipe, la planification d'équipe et personnelle.ainsi que la compréhension des outils collaboratifs.
Communication avec le module Lora	Apprentissage à l'utilisation du module,création du protocole et fiabilisation en solo
V_debug	Rédaction du code en cas de bug.
Saisie de schéma et routage	Saisie du schéma électrique et du routage donc création du module sous Proteus
Tests globaux	Test avec toutes les parties ainsi que test en condition semi-réelle

Figure 3: Tableaux des tâches prévisionnel

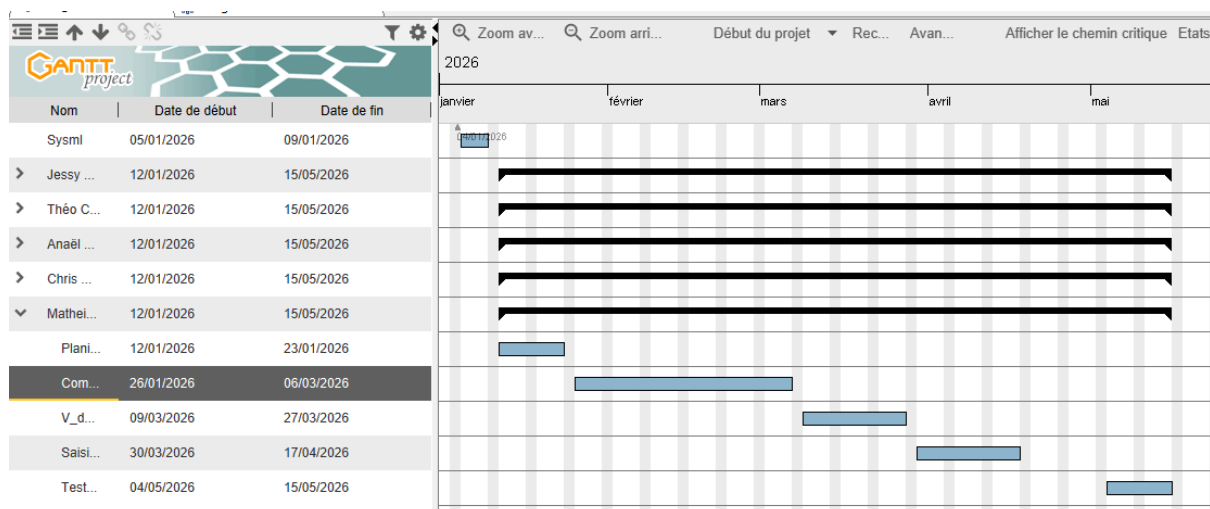


Figure 4: Planification prévisionnelle de la partie 3 + commun

J'ai prévu au départ de passer 2 semaines maximum sur la planification pour ne pas y aller trop rapidement tout en ne prenant pas trop de temps par sécurité.

Pour la partie "communication avec le module lora", je suis parti sur 6 semaines dans un premier temps, cela me paraissait suffisant car nous avions déjà travaillé avec ces modules dans le courant de l'année mais non pas sur des esp32 mais des module Wipy.

La version debug est prévu sur 3 semaines pour être sûr de la fiabilité de cette dernière.

La saisie de schéma et du routage était prévu sur 3 semaines pour pouvoir avoir un retour des professeurs et pouvoir s'échanger les schémas entre nous si besoin sans se presser pour autant.

Les tests avec les cartes officielles ont été prévu sur 2 semaines pour avoir le temps d'adapter les codes si besoins et de faire un test global en condition "réel".

2.2. Planification Réelle :

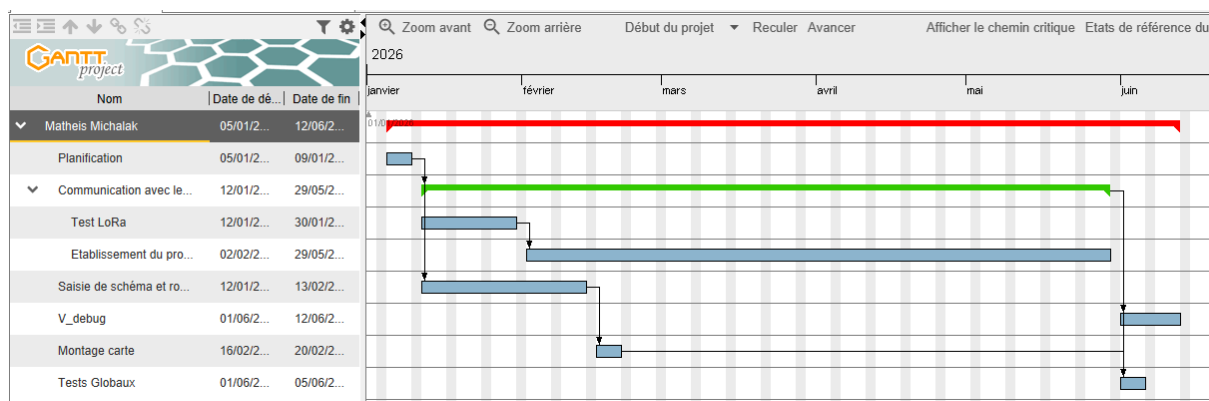


Figure 5: Planification réel et final du projet.

Finalement, la partie planification dans son intégralité n'a duré qu'une semaine car les logiciels étaient connus et que la grosse partie de cette semaine a été passée sur l'utilisation de GitHub.

Au contraire, la partie lora a rencontré plusieurs imprévus comme on peut le voir sur la figure ci-dessus. La tâche LoRa s'est effectuée en deux parties, une première partie test où j'ai appris à utiliser le module avec un

esp32 et une deuxième ou j'ai donc établi le protocole de la communication du projet.

La version debug est partie au second plan pour se consacrer pleinement au protocole avant toute chose.

La partie schéma et routage a finalement été effectuée en simultané de la partie communication car elles ne dépendent pas l'une de l'autre ,cela permettait aussi de pouvoir envoyer la carte en fabrication le plus tôt possible.

Les tests globaux se font en même temps que la version debug pour prendre de l'avance sur l'adaptation et dureront 1 semaine pour garder 1 semaine de libre avant les oraux.

3. Technologies Utilisé :

3.1. Pourquoi ce module ?

Le module **E32-900T30D** est un émetteur-récepteur sans fil à port série (**UART**) et non SPI ,comme il est couramment utilisé ,car la **liaison série** est plus facile à mettre en place avec un ESP32. Il utilise la technologie de modulation **LoRa** à spectre étalé, ce qui lui permet d'atteindre des distances de communication relativement élevées tout en offrant une forte immunité aux interférences.

3.2. Explications Techniques :

a. Caractéristiques Principales :

- **Distance de transmission** : Peut atteindre jusqu'à 8 km dans des conditions idéales (air libre, antenne gain 5dBi, hauteur 2,5m).
- **Fréquence** : Fonctionne sur la bande 862MHz à 931 MHz, supportant les bandes européennes standards 868/915MHz.
- **Puissance d'émission** : Jusqu'à 30 dBm (interdit pour amateur en france), ajustable par logiciel, utilisation sur 20-27 dBm.
- **Alimentation** : Entre 3,3V et 5,5V DC. Pour une performance optimale (puissance maximale), une tension de 5V ou plus est recommandée.

b. Modes de Fonctionnement :

Le module dispose de quatre modes de fonctionnement déterminés par l'état des broches **M0** et **M1**:

Mode	M1	M0	Description
0 Normal	0	0	Transmission transparente. Le port série et le canal sans fil sont ouverts.
1 Wake-up	0	1	Identique au mode normal, mais ajoute un préambule pour réveiller les récepteurs en mode 2.

2 - Power-saving	1	0	Port série fermé. Le module écoute périodiquement le signal de réveil (polling) pour économiser la batterie.
3 - Sleep	1	1	Mode veille pour la configuration. Utilisé pour lire ou modifier les paramètres du module via des commandes HEX.

c. Types de Transmission :

Le module supporte deux principaux types d'envoi de données :

- **Transmission transparente (Broadcasting)** : Le module envoie les données à tous les autres modules configurés sur le même canal et avec la même adresse (ou adresse universelle 0xFFFF).
- **Transmission fixe (Fixed-point)** : Permet d'envoyer des données à un module spécifique en spécifiant son adresse et son canal au début du paquet de données.

d. Indicateur AUX :

La broche AUX est utilisée pour indiquer l'état du module:

- Elle passe au niveau **bas (0)** lorsque le module est occupé (auto-test à l'allumage, transmission sans fil en cours ou tampon de données plein).
- Elle passe au niveau **haut (1)** lorsque le module est prêt à recevoir de nouvelles données sur son port série ou que la tâche en cours est terminée.

e. Sécurité et Fiabilité

- **FEC (Forward Error Correction)** : Le module intègre une correction d'erreur directe qui améliore la stabilité de la communication en cas d'interférences.
- **Chiffrement et compression** : Les données sont compressées et cryptées avant l'envoi pour assurer la confidentialité et l'efficacité de la transmission.

3.3. Création et Câblage d'un Module :

No.	Description	Direction	No.	Description	Direction
1	M0	Entrée	7	GND	Entrée
2	M1	Entrée	8	Fix Hole	NC
3	RX	Entrée	9	Fix Hole	NC
4	TX	Sortie	10	Fix Hole	NC
5	AUX	Sortie	11	Fix Hole	NC
6	VCC	Entrée			

Figure 6: Tableau des Pins d'un module

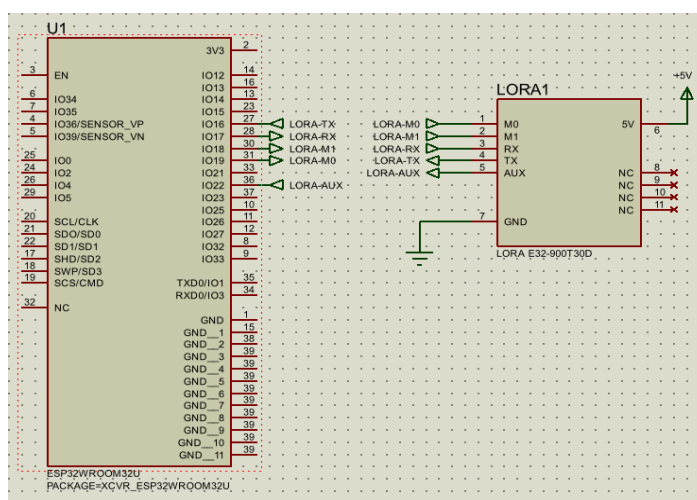


Figure 7: Schéma de câblage (lors de la Phase de développement - Proteus)

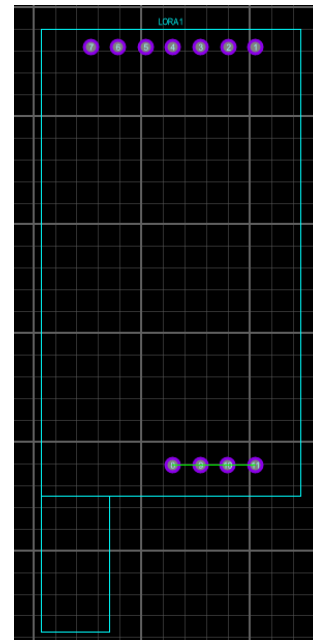
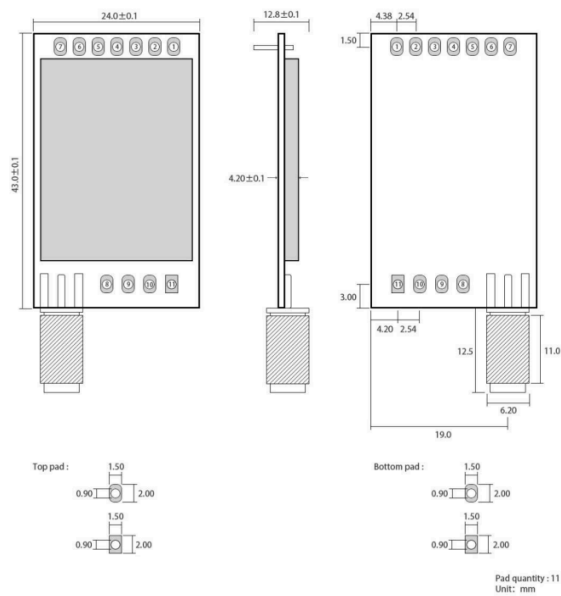


Figure 8: Côtes donnée par la datasheet et Empreinte de routage final (Proteus)

4. Objectif :

L'objectif principal de ma mission est de concevoir un protocole de communication basé sur une modulation LoRa, en utilisant le module E32-900T30D piloté via une liaison série. Ce travail englobe aussi bien la conception matérielle (hardware) que logicielle (firmware). Par conséquent, le câblage ainsi que le développement du code sont à réaliser.

Cette communication servira essentiellement à transmettre l'état de fonctionnement du système (données de diagnostic) ainsi que le niveau d'eau. Elle permet également d'envoyer des commandes au moteur pour lever ou abaisser la vanne selon les besoins. La liaison doit être la plus fiable possible afin d'éviter tout incident lié au débit d'eau. Pour garantir cette fiabilité, j'ai programmé un script de test basique en complément du programme principal.

Le module n'étant pas présent nativement dans la bibliothèque de Proteus et aucun modèle fiable n'étant disponible en ligne, j'ai dû procéder à sa création logicielle au sein de l'outil de CAO.

5. Principe de Communication :

5.1. Communication Simplex :

Ce mode de communication est établi sur le principe de base du module. C'est le premier type de communication que j'ai mis en place pour comprendre les réactions du module ainsi que la façon dont les informations sont relayées entre les modules.

Cette communication n'a pas besoin d'être synchronisée car seul l'un des modules émet et donc par conséquent il devient maître de la communication. Le principe est expliqué dans *l'algorithme 2.1 dans l'annexes*

5.2. Communication Full-Duplex :

La liaison full-duplex n'est pas gérée nativement par ces modules. Étant conçus pour du half-duplex, ils ont nécessité des ajustements complexes en raison de ce besoin spécifique de synchronisation. Dans le premier mode testé, la synchronisation était implicite, mais dans le cas présent, elle constitue le problème principal puisque la liaison attendue doit rester asynchrone. Le principe est expliqué dans *l'algorithme 2.2 dans l'annexes*

Le dysfonctionnement rencontré est le suivant : l'un des deux modules finit par basculer en mode « stand-by ». Par conséquent, la liaison est rompue car l'un des modules émet « dans le vide ».

J'ai d'abord supposé que cela pouvait provenir d'une baisse de tension ou d'un appel de courant trop important mettant le module en sécurité. Toutefois, les tests ont infirmé cette hypothèse : l'alimentation n'était pas la cause du problème. J'ai également vérifié le matériel de test afin d'exclure tout défaut de câblage ou de l'ESP32. Cette piste a été abandonnée après avoir remplacé les composants individuellement, puis l'intégralité du montage, sans succès.

5.3. Communication Half-Duplex (Final) :

5.3.1. Principe :

La liaison basée sur le principe du half-duplex nous permet de pallier le problème de synchronisation rencontré avec la liaison précédente.

Pour garantir un véritable échange d'informations, j'ai mis en œuvre un système de gestion en maître-esclave, où l'émission et la réception s'effectuent sur « demande » du maître.

5.3.2. Maître Pupitre :

Le pupitre mène la danse de la communication en émettant toutes les heures. Le but est de pouvoir avoir une communication programmée et se dédie à la réception par la suite .Le principe est expliqué dans *l'algorithme 2.3 dans l'annexes*

5.3.3. Esclave Barrage:

Ce code est le complémentaire du précédent. Le module esclave fonction en interruption uart-RX pour éviter la scrutation en attente du message .Le principe est expliqué dans *l'algorithme 2.4 dans l'annexes*

6. Élément de la Trame :

Pour les trames de la communication ,je me suis basé sur le principe des trames de gps : l'adresse du récepteur , le canal du récepteur, un caractère de début de trame , un caractère entre chaque information et un check à la fin pour valider la trame à l'arrivée, j'ai choisi le crc16 sur conseil de mes professeurs car le code est disponible en ligne et me fait donc gagner du temps pour travailler sur autre chose.

6.1. Trame d'Information :

Elle se présente sous la forme :

Add_msb+Addr_lsb+Canal+\$Nv_eau#Nv_vanne#État_système#Crc16

Chaque élément est récupéré depuis les parties de mes camarades, à l'exception du crc16 qui est généré à la création de la trame car il dépend justement des informations qu'il doit valider

Il y d'abord les informations demandées sous forme d'un dictionnaire qui sera analysé à la réception, ce dictionnaire est créé par la fonction `get_trame_info()` qu'elle retourne sous la forme suivante.

```
return "$"+str(water_level) + "#" + str(etat_vanne) + "#" + str(charge_level) + "#" + str(etat_systeme) + "#"
```

Ensuite il faut choisir le module a qui on veut envoyer les informations, donc il faut connaître ses octets d'adresses ainsi que son canal. Une fois connu on lance la fonction `lora.envoyer_trame(addr_h, addr_l, canal)`. Cette fonction lance elle-même la fonction `get_trame_info()` ainsi que la fonction `crc16()` qui prend la trame et retourne le crc correspondant. Tout cela est concaténé pour l'envoie sans oublier d'encodé en utf-8 la partie message global pour pouvoir la déchiffrer par la suite.

```
trame_emision = self.get_trame()          # Sous la forme: " $ val3 # val4 # texte2 # "  
crc = self.crc16(trame_emision)          # calcul du crc16 de la trame à émettre  
message = trame_emision + crc            # Sous la forme: " $ val1 # val2 # texte # crc16 "  
self.send_data(addr_high, addr_low, CANAL, message.encode('utf-8'))
```

7. Manipulation du Systèmes :

7.1. Règle de sécurité :

Le module E32-900t-30d est un appareil relativement résistant ,il à pu m'arriver de le laisser alimenter sans ESP32 connecter ou sans alimentation avec un ESP32 qui envoie sur la liaison série.

Malgré sa résistance ,il est conseillé ,comme avec tout appareil électronique ,de le câbler hors tension pour éviter les surprises.

Aussi ,mieux vaut ne pas lui envoyer une nouvelle configuration ou des messages à envoyer trop souvent. Durant les premiers tests en simplex ,un envoi toutes les secondes le fait chauffer mais toutes les trois secondes reste envisageable.

7.2. Opération de Contrôle :

7.2.1. Refus d'obtempérer :

Pour vérifier que le module est opérationnel, il faut inspecter la broche AUX, qui signale la disponibilité du module pour l'action demandée.

Si l'AUX reste à l'état haut malgré l'envoi de diverses commandes, cela signifie que le module s'est mis en sécurité. En revanche, s'il passe à l'état bas, cela indique que le module fonctionne correctement. En effet, le module signale via cette broche s'il est occupé à traiter une tâche ou s'il est libre.

7.2.2. Problème de synchronisation :

Il peut arriver qu'une trame soit invalide sans pour autant qu'une erreur soit signalée lors de l'envoi.

Le problème vient de la broche AUX : si celle-ci passe à 0 avant le début de la réception, aucun problème n'est à déclarer. En revanche, si l'AUX passe à 0 après le début de la réception, une partie de la trame peut être perdue.

7.3. Opération de Maintenance :

Il m'est arrivé durant le projet qu'un des deux modules ne veuille plus ni envoyer ,ni recevoir. Il faut donc si cela vous arrive : débrancher d'abord l'ESP et après l'alimentation du module , une fois rebranché le module reprend la configuration attribuée à la dernière sauvegarde et tout repart normalement.

Annexes:

1. Code sous MicroPython:

1.1. Création d'un module

```
12 # Cablage de l'ESP32 sur le module Lora
13 UART2 = 2 # deuxième port UART de l'ESP32
14 RX = 16
15 TX = 17
16 AUX = 22
17 M0 = 19
18 M1 = 18
19
20 # Configuration des modules LoRa - Envoi simple de données en point à point
21 # Adresse du module Lora sur le barrage
22 addr_high_barrage = 0x00
23 addr_low_barrage = 0x02
24 # Adresse du module Lora sur le pupitre de commande
25 addr_high_pupitre = 0x00
26 addr_low_pupitre = 0x03
27 # Canal de communication
28 CANAL = 0x06 # fréquence 868 MHz (canal 6)
29 DEBIT = 9600 # Débit communication série
30 CONF = 0xC0 # mode fixe à 30 dBm
31
32 #Instanciation de la liaison serie LORA-ESP32 avec interruption
33 uart = UART(UART2, baudrate=DEBIT, tx=TX, rx=RX)
34 #Instanciation du module Lora
35 lora_pupitre = LoRaE32('900T30D', uart, AUX, M0, M1)
36 lora_pupitre.begin()
37
38 # Configuration du module Lora
39 lora_pupitre.config['CHANNEL'] = CANAL # fréquence 868 MHz (canal 6)
40 lora_pupitre.config['ADDR_H'] = addr_high_pupitre # adresse pupitre MSB
41 lora_pupitre.config['ADDR_L'] = addr_low_pupitre # adresse pupitre LSB
42 lora_pupitre.config['OPTION'] = CONF # mode fixe à 30 dBm
43 lora_pupitre.set_config(save=True)
44
```

Dans l'ordre :

- Attribution des Pin physique de l'ESP en fonction du module (penser à intervertir le RX-TX)
- Adressage du Maître et de l'Esclave (penser a mettre les mêmes configuration sur les différents modules)
- Configuration des options de la communication
- Création de la liaison UART entre l'ESP et le module
- Création de l'entité "module lora" au niveau physique(nom variant selon la place du module dans la communication)
- Création de l'entité "module lora" au niveau système(nom variant selon la place du module dans la communication)

1.2. Main Maître et Interruption

```

47 reception_flag = False      # interruption RX valide si données séries reçues
48 emission_flag = False
49
50 # ***** SOUS PROGRAMMES INTERRUPTION *****
51 # interruption matérielle uart 2 :
52 def uart_handler(uart):
53     global reception_flag    # Pas de traitement long ici !
54     reception_flag = True    # Présence données reçues sur RX !
55
56 # interruption logicielle du timer 1 :
57 def timer_handler(timer_send_data):
58     global emission_flag     # Pas de traitement long ici !
59     emission_flag = True     # données à envoyer
60
61 # **** Activation des interruption ****
62 uart.irq(handler=uart_handler, trigger=UART.IRQ_RXIDLE) # Active IRQ sur RX
63 timer_send_data.init(mode=Timer.PERIODIC, period=5000, callback=timer_handler)
64

```

Dans l'ordre :

- Flag pour les différents handler
- Handler de la réception sur l'UART
- Handler de l'émission avec Timer
- Interruption matérielle sur la broche RX
- Interruption logicielle sur le Timer(1) alias timer_send_data toute les 5s

```

94 while True:
95     if emission_flag:
96         print(Color.G)
97         lora_pupitre.envoyer_trame(addr_high_barrage, addr_low_barrage, CANAL) #envoyer_trame(self, ad
98         compteur_trames_emises += 1 # On compte les trames envoyées
99         print(f"Nombre de trames envoyées par le module pupitre : {compteur_trames_emises}")
100        emission_flag = False
101        if reception_flag :
102            print(Color.Y)
103            trame = lora_pupitre.lecture_trame()
104            if isinstance(trame, dict):
105                for key in ['val1', 'val2', 'textel', 'cnc16']:
106                    print(f"\t * {key}: {trame[key]}") # On affiche les données reçues
107                    compteur_trames_valides += 1
108            else :
109                print("\t * trame non valide !") # trame non valide à rejeter
110                compteur_trames_invalides += 1
111            reception_flag = False # prochaines données en attente par flag interruption
112            print(Color.B)
113            print(f"Nombre de trames valides reçues du barrage : {compteur_trames_valides}")
114            print(f"Nombre de trames non valides reçues du barrage : {compteur_trames_invalides}")
115            time.sleep_ms(100)

```

Dans l'ordre :

- Check de l'interruption d'émission
- Envoie de la trame au module du barrage et mise à false du flag
- Check de l'interruption de réception
- Lecture et découpage de la trame pour analyse
- Validation ou invalidation de la trame reçu

1.3. Main Esclave et Interruption

```
45 # ***** SOUS PROGRAMMES INTERRUPTION *****
46 reception_flag = False # interruption RX valide si données séries reçues
47 emission_flag = False
48
49 # interruption matérielle uart 2 :
50 def uart_handler(uart):
51     global reception_flag # Pas de traitement long ici !
52     reception_flag = True # Présence données reçues sur RX !
53
54 # **** Activation des interruption ****
55 uart irq(handler=uart_handler, trigger=UART.IRQ_RXIDLE) # Active IRQ sur RX
56 |
```

Dans l'ordre :

- Flag pour le handler
- Handler de la réception sur l'UART
- Interruption matérielle sur la broche RX

```
85 while True:
86     if reception_flag :
87         print(Color.Y)
88         trame = lora_barrage.lecture_trame()
89         if isinstance(trame, dict):
90             for key in ['val1', 'val2', 'textel', 'crc16']:
91                 print(f"\t * (key): {trame[key]}") # On affiche les données reçues
92             compteur_trames_valides += 1
93             print(Color.G)
94             lora_barrage.envoyer_trame(addr_high_pupitre, addr_low_pupitre, CANAL) #envoyer_trame(self, addr_high, addr_low, CANAL);
95             compteur_trames_emises += 1 # On compte les trames envoyées
96             print(f"Nombre de trames envoyées par le module barrage : {compteur_trames_emises}")
97             print(Color.Y)
98         else :
99             print("\t * trame non valide !") # trame non valide à rejeter
100             compteur_trames_invalides += 1
101     reception_flag = False # prochaines données en attente par flag interruption
102     print(Color.B)
103     print(f"Nombre de trames valides reçues du barrage : {compteur_trames_valides}")
104     print(f"Nombre de trames non valides reçues du barrage : {compteur_trames_invalides}")
105     time.sleep_ms(100)
106
```

Dans l'ordre :

- Check de l'interruption de réception
- Lecture et découpage de la trame pour analyse
- Validation ou invalidation de la trame reçu
- Envoie de la trame au module du barrage et mise à false du flag
-

1.4. Dictionnaire de la Trame :

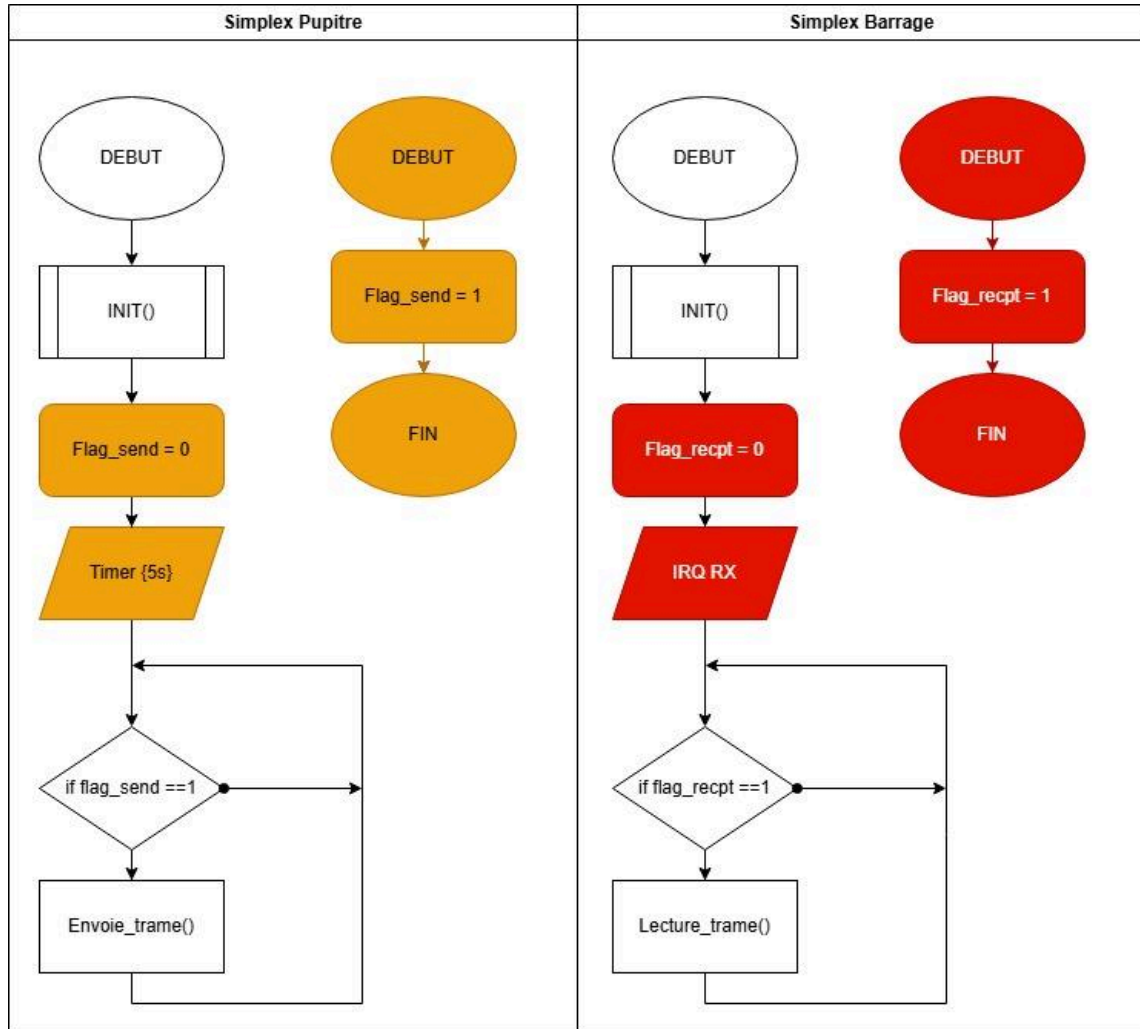
```
171     def get_trame_info(self): # Création d'une trame sous la forme: $ val3 # val4 # texte2 #
172         water_level = get_distance()
173         etat_vanne = get_state_vanne()
174         charge_level, _, _ = read_all_value()
175         etat_systeme = get_diag_system()
176         print(f"\t * Niveau d'Eau : {water_level}")
177         print(f"\t * Etat de la Vanne : {etat_vanne}")
178         print(f"\t * Charge de la Batterie : {charge_level}")
179         print(f"\t * Diagnostic du Système: {etat_systeme}")
180         return "$"+str(water_level) + "#" + str(etat_vanne) + "#" + str(charge_level) + "#" + str(etat_systeme) + "#"
```

Dans l'ordre :

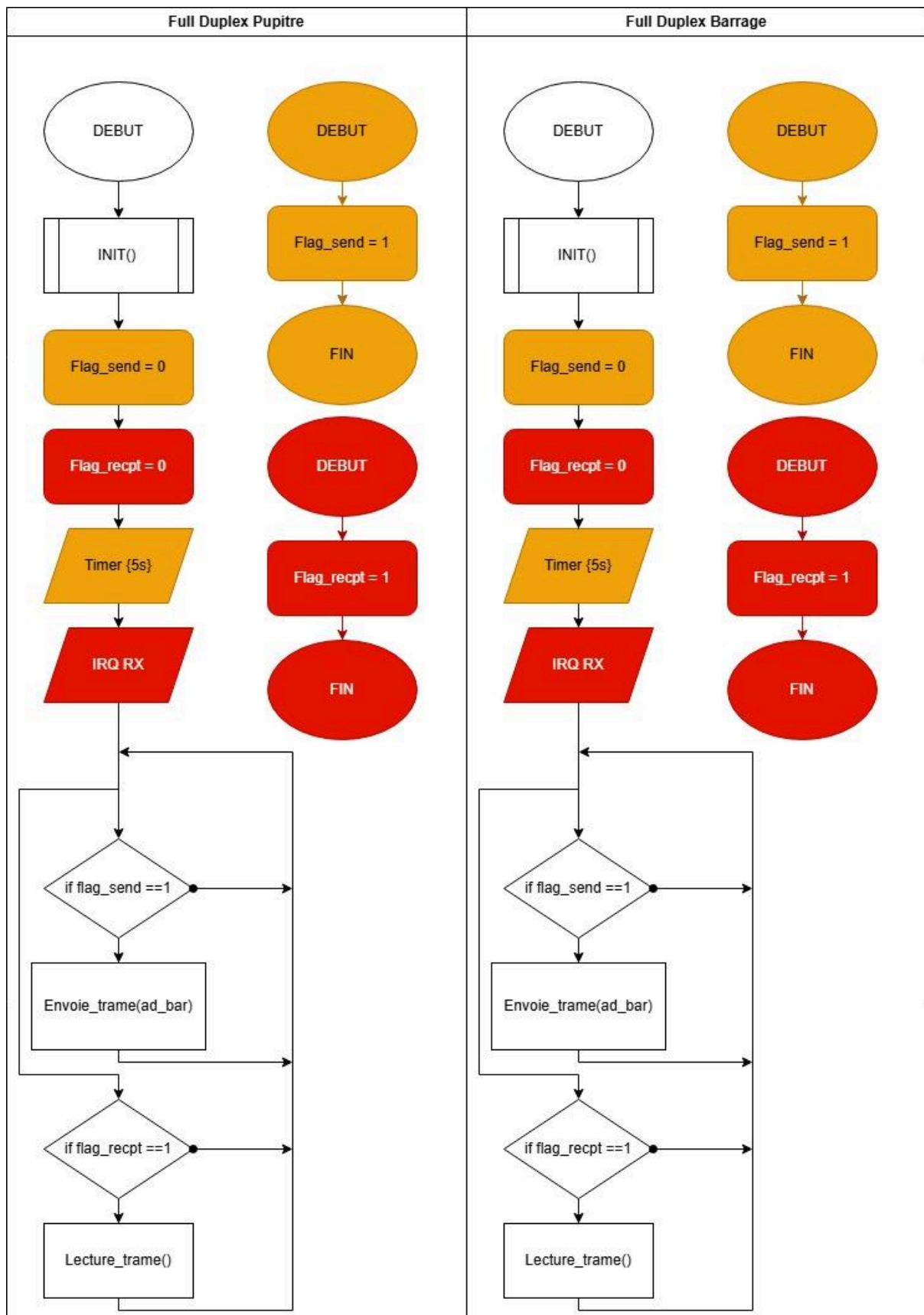
- Récupération de la hauteur d'eau
- Récupération de l'état de la vanne (haut ou bas)
- Récupération de la charge de la batterie
- Récupération du diagnostic du système
- Retour du dictionnaire complet prêt à l'envoi

2. Algorithme :

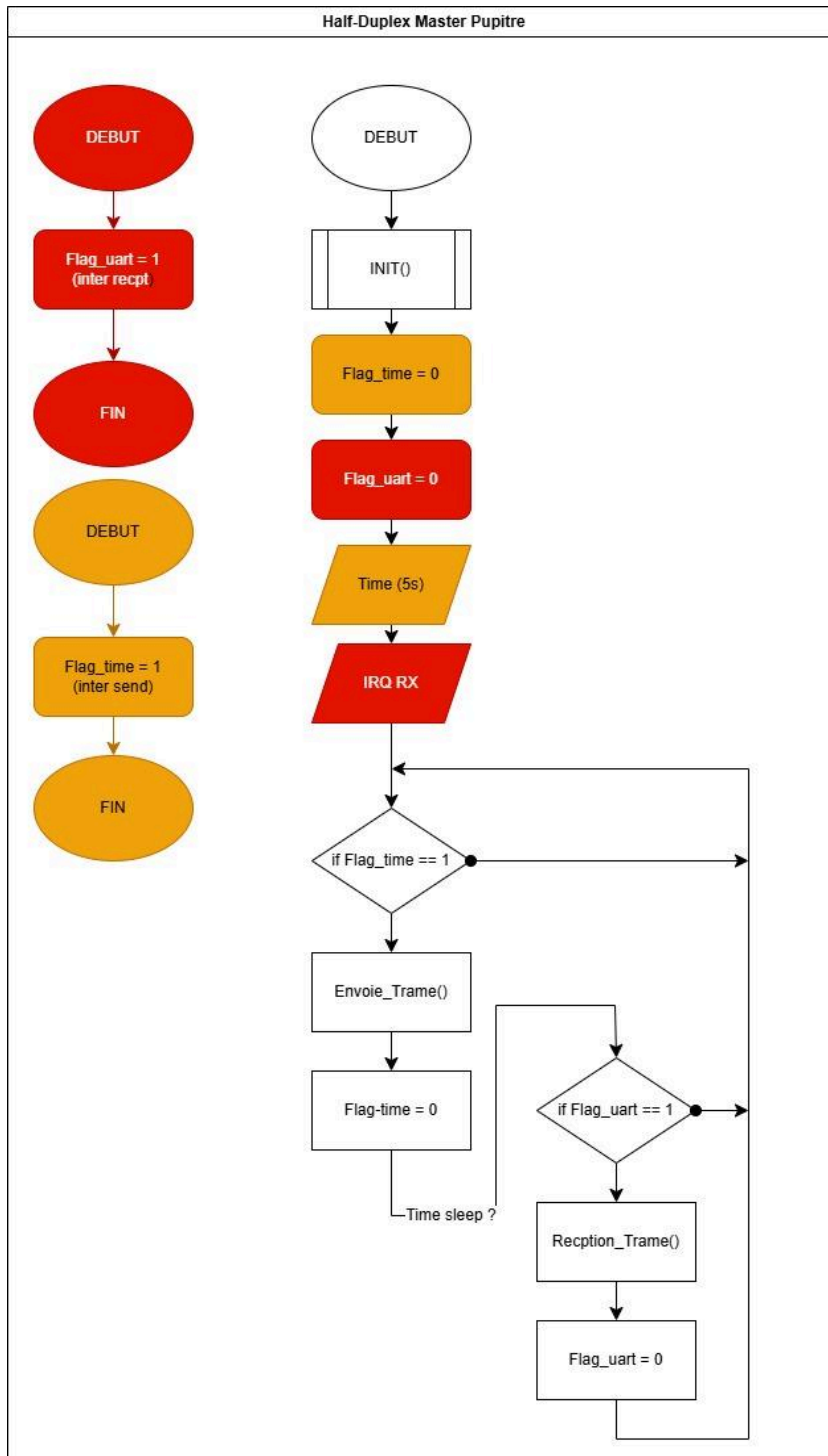
2.1. Algorithme Liaison Simplex :



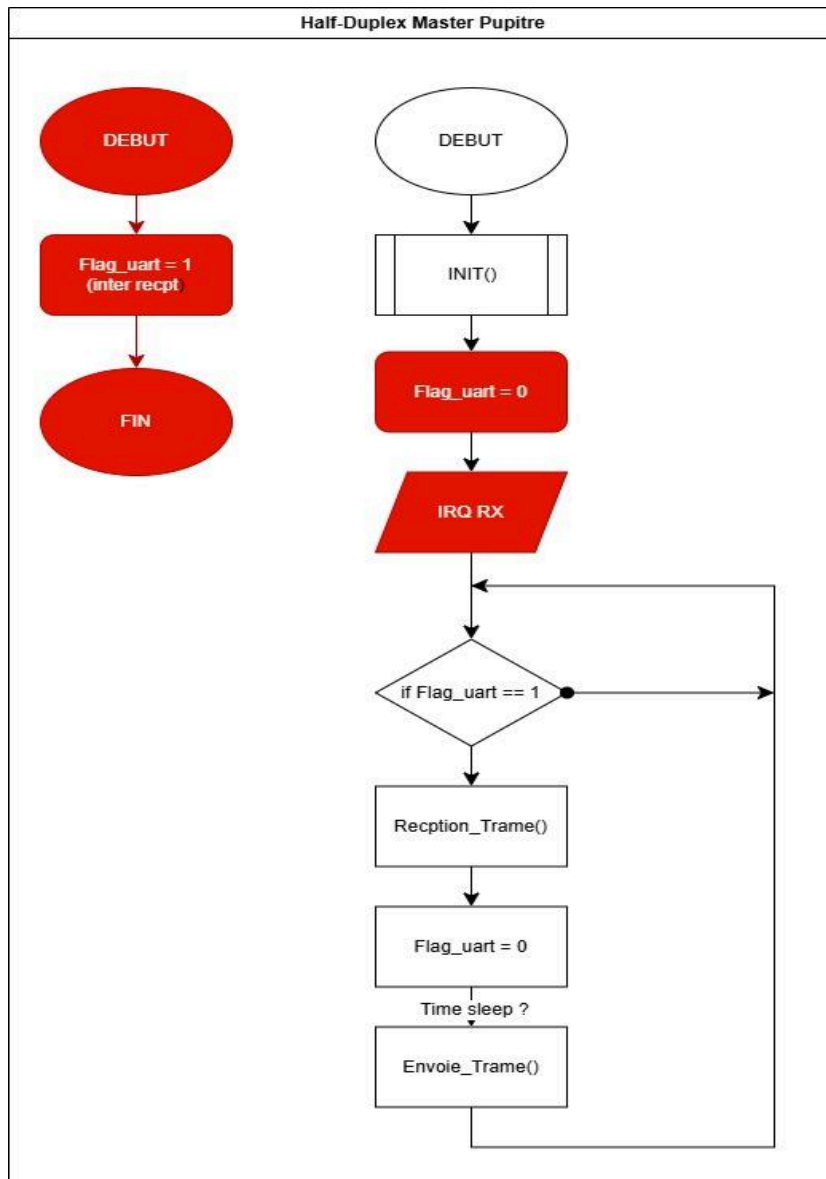
2.2. Algorithme Liaison Full-Duplex :



2.3. Algorithme Liaison Half-Duplex Maître Pupitre :

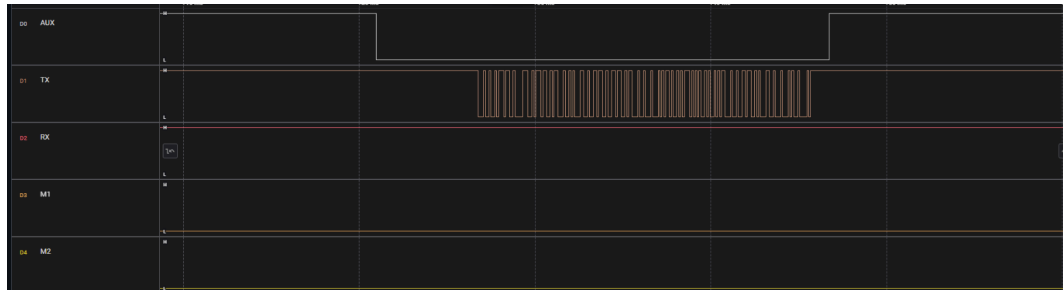


2.4. Algorithme Liaison Half-Duplex Esclave Barrage :



3. Gestion de la Trame :

3.1. Trame Valide :



3.2. Trame Invalide :

